SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: **HTTP Response Analysis**

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
```

```
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method URL Example Data Visibility Security Level GET /login?user=admin&pass=1234 Visible in URL Low POST /login Hidden in body Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why. 3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.

4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

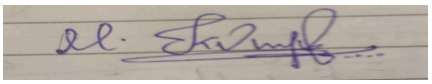
1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I KUMARAN M (192525022) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understandi ng of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

HTTP Response Analysis

1. Identify the incorrect or invalid fields in the header.

The given HTTP response header is:

```
http
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

The incorrect field is Content-Length: -150 because the content length must be a non-negative integer representing the size of the response body in bytes.
The other fields are generally valid. However, the date and the actual response body should also be verified.

2. Analyze the issue with Content-Length and explain its impact.

The Content-Length header specifies the size of the HTTP response body in bytes.

- A negative value such as -150 is invalid.
- It violates the HTTP message framing rules.
- The client may reject the response or report a parsing error.
- Incorrect message framing can cause communication problems between the client and server.

Therefore, the Content-Length value must be set to the actual size of the response body in bytes.

3. Determine whether the status code matches the response correctness.

The status code 200 OK indicates that the server has successfully processed the request.
However, the status code does not guarantee that every response header is valid.
In this example, the response contains an invalid Content-Length value. Therefore,

even though the status code is 200 OK, the response header is malformed.
The server should send a valid response with correct headers and an appropriate status code.

4. Suggest corrections for the identified errors.
The negative content length must be replaced with the actual non-negative size of the response body.
Corrected HTTP response:

http
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

Here, Content-Length: 150 is only an example. The actual value must match the response body size. The date should also match the actual response date; 1 April 2026 was a Wednesday, not a Monday.

Conclusion:
The main error in the given HTTP response is the negative Content-Length value. Correcting it and verifying the other header fields ensures proper HTTP message framing and more reliable communication between the client and server.

DOM Structure Analysis

Given DOM Structure

HTML

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```



1. Identify missing or improperly closed elements.

The given HTML code contains the following error:

- The element is not explicitly closed with .
- The element is properly closed with .
- The <html>, <head>, <title>, and <body> elements are properly closed.

Therefore, the main issue is the missing closing tag for the paragraph element.

2. Analyze how the browser will interpret this structure.

Browsers use the HTML parser to convert HTML code into a Document Object Model (DOM) tree.

- The browser detects the unclosed <p> element.
- When it encounters , the HTML parser automatically closes the paragraph element in this situation.
- The browser attempts to recover from the malformed HTML instead of stopping the page from loading.
- The resulting DOM generally contains the paragraph inside the element.

Thus, the browser can display the page even though the source HTML is not properly written.

3. Determine the impact on rendering and layout.

The missing closing tag may have the following effects:

- The browser may automatically correct the HTML structure.
- The text HelloWorld will generally appear inside the paragraph.
- Unexpected nesting or layout issues may occur in more complex HTML documents.
- CSS styles and JavaScript operations may behave unexpectedly if the resulting DOM differs from the developer's intention.
- Different malformed structures can lead to different browser-recovery results.

In this simple example, the visible impact is likely minimal because browsers handle this error automatically.

4. Suggest corrections to make the DOM valid.

The paragraph element should be explicitly closed before the closing tag.
Corrected HTML code:

HTML

```
<!DOCTYPE html>
<html>
<head>
   <title>TestPage</title>
</head>
<body>
```

```
    <div>
        <p>HelloWorld</p>
    </div>
</body>
</html>
```



Explanation:

- <!DOCTYPE html> declares the HTML5 document type.
- starts the paragraph.
- properly closes the paragraph.
- closes the division element.
- All other elements are properly nested and closed.

Conclusion:

The main error in the given DOM structure is the missing tag. Browsers generally recover by automatically closing the paragraph when they encounter . Explicitly closing all elements improves code readability, maintainability, and predictable rendering.

GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

GET and POST are HTTP methods used to send data between a client and a server.

GET Method	POST Method
Sends data through the URL.	Sends data in the request body.
Data is visible in the URL.	Data is not normally visible in the URL.
Can expose sensitive information in browser history and logs.	Reduces exposure through URLs, but does not encrypt data.
Mainly used to retrieve information.	Commonly used to submit data.
Not suitable for sending login credentials.	More suitable for submitting login credentials.

2. Analyze which method is more suitable for login and why.

POST is more suitable for login authentication because the username and password are sent in the request body rather than in the URL.

Reasons:

- Credentials are not directly displayed in the URL.
- It reduces the risk of credentials appearing in browser history and URL logs.
- It is suitable for submitting authentication forms.
- It supports sending form data to the server for verification.

However, POST alone does not provide encryption. HTTPS must be used to protect credentials during transmission.

3. Identify security risks in using GET for authentication.

Using GET for login can create several security risks:

1. URL exposure: Passwords may be visible in the address bar.
2. Browser history: Credentials may remain in browsing history.
3. Server logs: URLs may be recorded in server logs.
4. Shared systems: Other people may see credentials on shared computers.

5. Data leakage: URLs may be copied, bookmarked, or exposed through other systems.

Therefore, GET should not be used to transmit passwords or other sensitive authentication information.

4. Suggest best practices for secure login implementation.

The following practices help protect login systems:

1. Use POST: Send login credentials in the request body.
2. Use HTTPS: Encrypt communication between the client and server.
3. Hash passwords: Store passwords using secure password-hashing algorithms such as Argon2id or bcrypt.
4. Validate input: Validate credentials on the server side.
5. Use secure sessions: Set appropriate HttpOnly, Secure, and SameSite cookie attributes.
6. Limit login attempts: Apply rate limiting to reduce password-guessing attacks.
7. Avoid revealing details: Use generic error messages such as “Invalid username or password.”
8. Use multi-factor authentication: Add another verification step when appropriate.

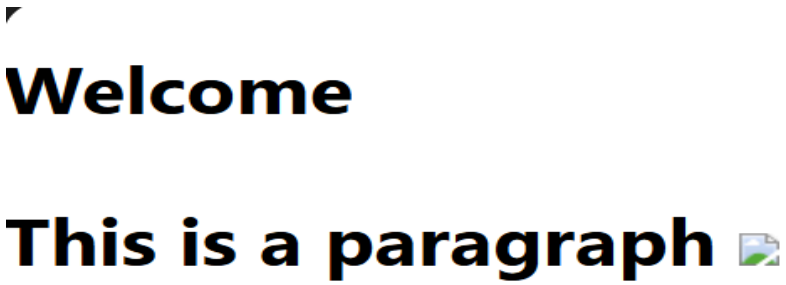
Conclusion:

POST is the preferred HTTP method for login systems because it sends credentials in the request body. However, POST does not encrypt data by itself. A secure login system must combine POST with HTTPS, secure password storage, server-side validation, and session protection.

HTML Code Inspection

Given HTML Code

HTML
```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```



1. Identify missing closing tags.

The following errors are identified in the given HTML code:

- The <h1> heading tag is not explicitly closed with </h1>.
- The paragraph tag is not explicitly closed with .
- The <img> tag does not require a closing tag because it is a void element in HTML.
- The remaining tags, including <html>, <head>, <title>, and <body>, are

properly closed.

2. Analyze issues with the <img> tag.
The image tag is written as:
HTML
<img src="image.jpg">
The following points should be considered:

- The src attribute specifies the image file path.
- The alt attribute is missing. It provides alternative text if the image cannot be displayed and supports screen readers.
- The image will not appear if image.jpg is missing or the path is incorrect.
- The <img> tag is valid without a closing tag in HTML5.

3. Determine how browsers will handle these errors.
Browsers use an HTML parser to interpret the code and construct the DOM tree.

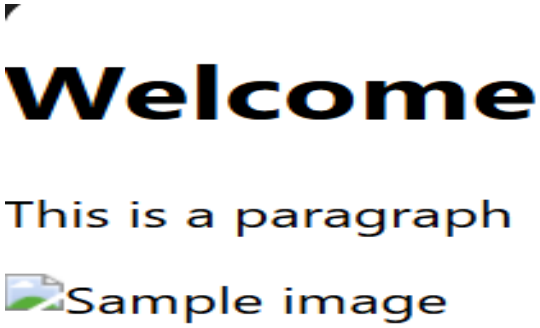
- The browser generally infers the end of the <h1> element when it encounters the <p> start tag.
- The paragraph is automatically closed when the browser reaches the end of the document.
- The image is displayed if the specified file exists and can be loaded.
- The browser attempts to recover from these errors rather than stopping the page from loading.

Therefore, the page may display correctly despite the missing closing tags, but the source code should be corrected for clarity and reliability.

4. Suggest a corrected version of the code.
Corrected HTML code:

HTML
<!DOCTYPE html>
<html>
<head>

```
    <title>Sample</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph</p>
    <img src="image.jpg" alt="Sample image">
</body>
</html>
```



Corrections made:

1. Added </h1> to close the heading.
2. Added </p> to close the paragraph.
3. Added alt="Sample image" to improve accessibility.
4. Kept the <img> tag without a closing tag because it is a void element.

Conclusion:

The main issues in the given HTML code are the missing closing tags for the heading and paragraph and the missing alt attribute in the image tag. Correcting these issues improves code readability, accessibility, and maintainability.

Browser–Server Communication Flow

1. Interpret each step in the communication flow.

Browser–server communication is the process through which a browser requests a web page from a server and displays the received content.

Step 1: User enters a URL in the browser

- The user enters a website address, such as www.google.com.
- The browser identifies the destination website.

Step 2: DNS lookup occurs

- DNS stands for Domain Name System.
- It converts the domain name into an IP address.
- The IP address helps the browser locate the server.

Step 3: HTTP request is sent

- The browser establishes a connection with the server.
- It sends an HTTP or HTTPS request to access the required resource.
- The request may include headers, cookies, and other information.

Step 4: Server processes the request

- The server receives and analyzes the request.
- It may execute application code, access a database, and prepare the requested content.

Step 5: Response is returned

- The server sends an HTTP response containing a status code, headers, and possibly a response body.
- For example, 200 OK indicates a successful request.

Step 6: Browser renders the page

- The browser processes HTML, CSS, and JavaScript.
- It loads additional resources such as images and fonts.
- Finally, it displays the web page to the user.

2. Identify where delays can occur.

Delays can occur at different stages of communication:

1. DNS lookup: The DNS server may respond slowly.
2. Connection establishment: Creating a TCP connection and establishing TLS for HTTPS may take time.
3. Network transmission: Slow internet speed or network congestion can delay requests.
4. Server processing: Complex operations or slow database queries can increase response time.
5. Response transfer: Large files and slow connections can delay downloading.
6. Browser rendering: Large images, complex CSS, and JavaScript execution can slow page display.

3. Analyze the role of DNS in the process.

DNS plays an important role in browser–server communication by translating human-readable domain names into IP addresses.

For example:

www.example.com → IP address

Its main functions are:

- Resolves domain names into IP addresses.
- Helps the browser locate the correct server.
- Supports the use of domain names instead of remembering numerical IP addresses.
- Uses caching to reduce repeated DNS lookups and improve performance.

Without DNS, users would generally need to know the server's IP address to access a website by address.

4. Suggest optimizations to improve performance.

The following techniques can improve website performance:

1. DNS caching: Store DNS results temporarily to reduce repeated lookups.
2. Use a CDN: A Content Delivery Network serves resources from servers closer to users.
3. Optimize images: Compress images and use suitable formats such as WebP or AVIF.
4. Enable browser caching: Store static resources locally for future visits.
5. Optimize server code: Improve application logic and database queries.
6. Minify files: Reduce the size of HTML, CSS, and JavaScript files.
7. Use HTTP/2 or HTTP/3: Improve the efficiency of transferring multiple resources.
8. Reduce unnecessary requests: Remove unused scripts and resources.
9. Use compression: Enable Brotli or Gzip where appropriate.
10. Optimize rendering: Defer non-critical JavaScript and load off-screen images lazily.

Conclusion:

Browser–server communication involves URL entry, DNS resolution, request transmission, server processing, response delivery, and browser rendering. Delays may occur at any stage. Using DNS caching, CDNs, optimized resources, efficient server processing, and browser caching can reduce loading time and improve the user experience.